

```
1 package strategy;
2
3 public interface Strategy {
4     int execute(int a, int b);
5
6     default void info(){
7         System.out.println("Выберите операцию");
8     }
9 }
```

```
1 package strategy;
2
3 public class Context {
4     Strategy strategy;
5
6     public void setStrategy(Strategy strategy) {
7         this.strategy = strategy;
8     }
9
10    public int executeStrategy(int a, int b) {
11        if (strategy != null) {
12            return strategy.execute(a, b);
13        } else {
14            throw new IllegalStateException("Strategy not set");
15        }
16    }
17 }
```

```
1 package strategy;
2
3 public class ConcreteStrategySubtract implements Strategy {
4     @Override
5     public int execute(int a, int b) {
6         return a - b;
7     }
8 }
```

```
1 package strategy;
2
3 public class ConcreteStrategyMultiply implements Strategy {
4     @Override
5     public int execute(int a, int b) {
6         if (a == 0 || b == 0) {
7             return 0;
8         } else {
9             return a * b;
10        }
11    }
12 }
```

```
1 package strategy;
2
3 public class ConcreteStrategyAdd implements Strategy {
4     @Override
5     public int execute(int a, int b) {
6         return a + b;
7     }
8 }
```

```
1 package App;
2 import strategy.*;
3
4 public class Strat {
5     Run | Debug
6     public static void main(String[] args) throws Exception {
7         int a = 1;
8         int b = 4;
9         char action = '+';
10
11         Context context = new Context();
12
13         if(action == '+'){
14             context.setStrategy(new ConcreteStrategyAdd());
15         }
16
17         if(action == '-'){
18             context.setStrategy(new ConcreteStrategySubtract());
19         }
20
21         if(action == '*'){
22             context.setStrategy(new ConcreteStrategyMultiply());
23         }
24
25         int result = context.executeStrategy(a, b);
26
27         System.out.println("result: " + result);
28     }
29 }
```

```
1 package chainOfResponsibility;
2
3 public abstract class Handler {
4     protected Handler next;
5
6     public void setNext(Handler next) {
7         this.next = next;
8     }
9
10    public abstract void handle(Request request);
11 }
```

```
1 package chainOfResponsibility;
2
3 public class FirstHandler extends Handler{
4     @Override
5     public void handle(Request request) {
6         if (request.getDeep() <= 1) {
7             System.out.println("First handler: processed the request.");
8         } else if (next != null) {
9             System.out.println("First handler: Передаю дальше");
10            next.handle(request);
11        }
12    }
13 }
```

```
1 package chainOfResponsibility;
2
3 public class SecondHandler extends Handler {
4     @Override
5     public void handle(Request request) {
6         if (request.getDeep() > 1 && request.getDeep() <= 2) {
7             System.out.println("Second handler: processed the request.");
8         } else if (next != null) {
9             System.out.println("Second handler: Передаю дальше");
10            next.handle(request);
11        }
12    }
13 }
```

```
1 package chainOfResponsibility;
2
3 public class ThirdHandler extends Handler {
4     @Override
5     public void handle(Request request) {
6         if (request.getDeep() > 2 && request.getDeep() <= 3) {
7             System.out.println("Third handler: processed the request.");
8         } else if (next != null) {
9             System.out.println("Third handler: Некому передавать дальше");
10            next.handle(request);
11        } else {
12            System.out.println("Запрос не был обработан: " + request.getDeep());
13        }
14    }
15 }
```

```
git / astor-lessons / module_5 / src / chainOfResponsibility
1 package chainOfResponsibility;
2
3 public class Request {
4     private int deep;
5
6     public Request(int deep) {
7         this.deep = deep;
8     }
9
10    public int getDeep() {
11        return deep;
12    }
13 }
```

```
git / astor-lessons / module_3 / src / App / Chain.java / Chain / main(String[])
1  package App;
2  import chainOfResponsibility.*;
3
4  public class Chain {
    Run | Debug
5      public static void main(String[] args) {
6          Handler handler1 = new FirstHandler();
7          Handler handler2 = new SecondHandler();
8          Handler handler3 = new ThirdHandler();
9
10         handler1.setNext(handler2);
11         handler2.setNext(handler3);
12
13         Request hand1 = new Request(deep: 1);
14         Request hand2 = new Request(deep: 2);
15         Request hand3 = new Request(deep: 3);
16         Request hand4 = new Request(deep: 4);
17
18         System.out.println("=== Обработка запроса 1 ===");
19         handler1.handle(hand1);
20
21         System.out.println("\n=== Обработка запроса 2 ===");
22         handler1.handle(hand2);
23
24         System.out.println("\n=== Обработка запроса 3 ===");
25         handler1.handle(hand3);
26
27         System.out.println("\n=== Обработка запроса 4 ===");
28         handler1.handle(hand4);
29     }
30 }
```

```
git / aston-lessons / module_3 / src / builder /  
1 package builder;  
2  
3 public interface Builder {  
4     void buildMotherboard();  
5     void buildProcessor();  
6     void buildRam();  
7     void buildSsd();  
8     void buildGpu();  
9     void buildWifi();  
10  
11     Computer getComputer();  
12  
13 }
```

```
git / aston-lessons / module_3 / src / builder / Computer.java / builder  
1 package builder;  
2  
3 public class Computer {  
4     private String motherboard;  
5     private String processor;  
6     private String ram;  
7     private String ssd;  
8     private String gpu;  
9     private boolean hasWifi;  
10  
11     public String getMotherboard() { return motherboard; }  
12     public String getProcessor() { return processor; }  
13     public String getRam() { return ram; }  
14     public String getSsd() { return ssd; }  
15     public String getGpu() { return gpu; }  
16     public boolean hasWifi() { return hasWifi; }  
17  
18     public void setMotherboard(String motherboard) { this.motherboard = motherboard; }  
19     public void setProcessor(String processor) { this.processor = processor; }  
20     public void setRam(String ram) { this.ram = ram; }  
21     public void setSsd(String ssd) { this.ssd = ssd; }  
22     public void setGpu(String gpu) { this.gpu = gpu; }  
23     public void setHasWifi(boolean hasWifi) { this.hasWifi = hasWifi; }  
24  
25     @Override  
26     public String toString() {  
27         return "Computer{" +  
28             "motherboard='" + motherboard + '\'' +  
29             ", processor='" + processor + '\'' +  
30             ", ram='" + ram + '\'' +  
31             ", ssd='" + ssd + '\'' +  
32             ", gpu='" + gpu + '\'' +  
33             ", hasWifi=" + hasWifi + '}';  
34     }  
35 }
```

```
1 package builder;
2
3 public class OfficeComputerBuilder implements Builder {
4     private Computer computer;
5
6     public OfficeComputerBuilder() {
7         this.computer = new Computer();
8     }
9
10    @Override
11    public void buildMotherboard() {
12        computer.setMotherboard(motherboard: "ASUS ROG Strix");
13    }
14
15    @Override
16    public void buildProcessor() {
17        computer.setProcessor(processor: "Intel Core i9-13900HX");
18    }
19
20    @Override
21    public void buildRam() {
22        computer.setRam(ram: "32 GB");
23    }
24
25    @Override
26    public void buildSsd() {
27        computer.setSsd(ssd: "1 TB");
28    }
29
30    @Override
31    public void buildGpu() {
32        computer.setGpu(gpu: "NVIDIA GeForce RTX 4090");
33    }
34
35    @Override
36    public void buildWifi() {
37        computer.setHasWifi(hasWifi: true);
38    }
39
40    @Override
41    public Computer getComputer() {
42        return computer;
43    }
44
45 }
```

git / astor-lessons / module_3 / src / Builder / Director.java / () Builder

```
1 package builder;
2
3 public class Director {
4     public Computer construct(Builder builder) {
5         builder.buildMotherboard();
6         builder.buildProcessor();
7         builder.buildRam();
8         builder.buildSsd();
9         builder.buildGpu();
10        builder.buildWifi();
11        return builder.getComputer();
12    }
13 }
```

git / astor-lessons / module_3 / src / App / Build.java / ...

```
1 package App;
2 import builder.*;
3
4 public class Build {
5     Run | Debug
6     public static void main(String[] args) {
7         Director director = new Director();
8
9         Builder officeBuilder = new OfficeComputerBuilder();
10        Computer offiComputer = director.construct(officeBuilder);
11        System.out.println("Office computer:");
12        System.out.println(offiComputer);
13    }
14 }
```

git / aston-lessons / module_3 / src / proxy

```
1 package proxy;
2
3 public interface Video {
4     void play();
5 }
6
```

git / aston-lessons / module_3 / src / proxy / RealVideo.java / () proxy

```
1 package proxy;
2
3 public class RealVideo implements Video {
4     private String filename;
5
6     public RealVideo(String filename) {
7         this.filename = filename;
8         loadVideo();
9     }
10
11     private void loadVideo() {
12         System.out.println("**Загрузка видео: " + filename);
13
14         try {
15             Thread.sleep(2000);
16         } catch (InterruptedException e) {
17             Thread.currentThread().interrupt();
18         }
19
20         System.out.println("--Видео '" + filename + "' загружено");
21     }
22
23     @Override
24     public void play() {
25         System.out.println("Воспроизводится видео: " + filename);
26     }
27 }
28
```

git / astor-lessons / module_3 / src / App / Proxy.java / ..

```
1 package App;
2 import proxy.*;
3
4 public class Proxy {
5     Run | Debug
6     public static void main(String[] args) {
7         Video video = new VideoProxy(filename: "GoF-patterns.mp4", userRole: "user");
8
9         System.out.println("Видео ещё не загружено");
10
11         video.play();
12
13         System.out.println("\n=== Доступ с ограниченной ролью ===");
14         Video restricteeVideo = new VideoProxy(filename: "GoF-patterns.mp4", userRole: "guest");
15         restricteeVideo.play();
16     }
17 }
```

git / astor-lessons / module_3 / src / decorator /

```
1 package decorator;
2
3 public interface Bevarage {
4     String getDescription();
5     double getCost();
6
7 }
```

git / astor-lessons / module_3 / src / decorator / SimpleCoffee.java / ..

```
1 package decorator;
2
3 public class SimpleCoffee implements Bevarage {
4
5     @Override
6     public String getDescription() {
7         return "Simple Coffee";
8     }
9
10    @Override
11    public double getCost() {
12        return 1.0;
13    }
14 }
```

git / astor-lessons / module_3 / src / decorator / BevarageDecorator.java / ...

```
1 package decorator;
2
3 public abstract class BevarageDecorator implements Bevarage{
4     protected Bevarage bevarage;
5
6     public BevarageDecorator(Bevarage bevarage) {
7         this.bevarage = bevarage;
8     }
9 }
```

git / astor-lessons / module_3 / src / decorator / ChocolateDecorator.java / ...

```
1 package decorator;
2
3 public class ChocolateDecorator extends BevarageDecorator {
4     public ChocolateDecorator(Bevarage bevarage) {
5         super(bevarage);
6     }
7
8     @Override
9     public String getDescription() {
10         return bevarage.getDescription() + ", Chocolate";
11     }
12
13     @Override
14     public double getCost() {
15         return bevarage.getCost() + 0.8;
16     }
17 }
```

```
1 package decorator;
2
3 public class MilkDecorator extends BevarageDecorator {
4
5     public MilkDecorator(Bevarage bevarage) {
6         super(bevarage);
7     }
8
9     @Override
10    public String getDescription() {
11        return bevarage.getDescription() + ", Milk";
12    }
13
14    @Override
15    public double getCost() {
16        return bevarage.getCost() + 0.5;
17    }
18 }
19
```

```
1 package decorator;
2
3 public class SugarDecorator extends BevarageDecorator {
4     public SugarDecorator(Bevarage bevarage) {
5         super(bevarage);
6     }
7
8     @Override
9     public String getDescription() {
10        return bevarage.getDescription() + ", Sugar";
11    }
12
13    @Override
14    public double getCost() {
15        return bevarage.getCost() + 0.2;
16    }
17 }
```

git / astor-lessons / module_3 / src / App / Decorator.java / ...

```
1 package App;
2 import decorator.*;
3
4 public class Decorator {
5     public static void main(String[] args) {
6         Bevarage coffee = new SimpleCoffee();
7         System.out.println(coffee.getDescription() + ": " + coffee.getCost());
8
9         Bevarage coffeeWithMilk = new MilkDecorator(new SimpleCoffee());
10        System.out.println(coffeeWithMilk.getDescription() + ": " + coffeeWithMilk.getCost());
11
12        Bevarage coffeeFancy = new ChocolateDecorator(
13            new SugarDecorator(
14                new MilkDecorator(
15                    new SimpleCoffee())));
16
17        System.out.println(coffeeFancy.getDescription() + ": " + coffeeFancy.getCost());
18    }
19 }
```

git / astor-lessons / module_3 / src / adapter / WeatherService.java / ...

```
1 package adapter;
2
3 public interface WeatherService {
4     /* Температура в Цельсиях
5     */
6     int getTemperature();
7     boolean isSunny();
8 }
```

git / astor-lessons / module_3 / src / adapter / WeatherDisplay.java / ...

```
1 package adapter;
2
3 public class WeatherDisplay {
4     private WeatherService service;
5
6     public WeatherDisplay(WeatherService service) {
7         this.service = service;
8     }
9
10    public void show() {
11        int temperature = service.getTemperature();
12        String weather = service.isSunny() ? "солнечно" : "облачно";
13        System.out.println("Температура: " + temperature + "*C, погода: " + weather);
14    }
15 }
```

git / astor-lessons / module_5 / src / adapter / ModernWeatherAPI.java / ...

```
1 package adapter;
2
3 public class ModernWeatherAPI {
4     public int getTempFahrenheit() {
5         return 68;
6     }
7
8     public String getCondition() {
9         return "sunny";
10    }
11 }
12
```

git / astor-lessons / module_5 / src / adapter / WeatherAdapter.java / ...

```
1 package adapter;
2
3 public class WeatherAdapter implements WeatherService{
4     private ModernWeatherAPI modernAPI;
5
6     public WeatherAdapter(ModernWeatherAPI modernAPI) {
7         this.modernAPI = modernAPI;
8     }
9
10    @Override
11    public int getTemperature() {
12        int fahrenheit = modernAPI.getTempFahrenheit();
13        return (int) ((fahrenheit - 32) * 5.0 / 9.0);
14    }
15    @Override
16    public boolean isSunny() {
17        return "sunny".equalsIgnoreCase(modernAPI.getCondition());
18    }
19 }
20
```

```
1 package App;
2
3 import adapter.*;
4
5 public class Adapter {
6     Run | Debug
7     public static void main(String[] args) {
8         WeatherService oldService = new WeatherService() {
9             @Override
10             public int getTemperature() {
11                 return 15;
12             }
13
14             @Override
15             public boolean isSunny() {
16                 return false;
17             }
18         };
19
20         System.out.println("Старый сервис:");
21         WeatherDisplay display1 = new WeatherDisplay(oldService);
22         display1.show();
23
24         ModernWeatherAPI modernAPI = new ModernWeatherAPI();
25         WeatherService adapter = new WeatherAdapter(modernAPI);
26
27         System.out.println("\nчерез адаптер и новый API: ");
28         WeatherDisplay display2 = new WeatherDisplay(adapter);
29         display2.show();
30     }
31 }
```